Overall, our model is looking good! If we're satisfied with the accuracy and professionalism scores, we mark this model as ready for production by giving it a production alias.

```
client = MlflowClient()
client.set_registered_model_alias(name=model_name, alias="Production",
version=model_version_to_evaluate)
```

Figure 7.60 – Aliasing our model to show it is production-ready

We evaluated our chatbot by creating predictions from our question-and-answer dataset, created a custom evaluation metric to evaluate the professionalism of each response, and visualized information about our model outputs. In the final chapter, we will demonstrate how to build a Gradio app so that you can bring your RAG chatbot to your end users!

Summary

Implementing models in production can be challenging, but with tools designed to support productionizing models and automating the MLOps process, it is much easier. In this chapter, we looked at using the Unity Catalog Model Registry to manage the lifecycle of a machine learning model. We showcased MLflow and how it can be used to create reproducible, modularized data science workflows that automatically track parameters and performance metrics. We discussed techniques for calculating features at the time of inference. To make the end-to-end MLOps process more manageable, we showed how to use workflows and webhooks to automate the ML Lifecycle. We also showed how to serve models and make inferences using the MLFlow and the Databricks Platform.

In the final chapter, *Monitoring, Evaluating, and More*, we will look at monitoring our data and machine learning models within the Databricks Lakehouse so you can get the most value from your data.

Questions

Let's test ourselves on what we've learned by going through the following questions.

1. Can more than one model be in production simultaneously? When would you want to use two models in production?
2. What component of MLflow could you use to route approvals?
3. Can you use an MLflow API to serve your model?